

오늘의 작업기록 — 2026년 7월 15일

에러 응답 JSON 구조화 — record 직렬화(마법 ④ 역방향) + 편집환경 시험

(에러 본문이 자유 문자열에서 {code, message} JSON이 된 날 — 그리고 폰으로 코드 편집할 다리를 놓은 날)

한눈에

항목	내용
목표	조각 2(어제 이월): 에러 응답 JSON 구조화 · 별건: code-server 편집환경 시험
결과	조각 2 완주 ✓ (3핸들러 JSON화, 4연타 통과) · code-server 시험 성공 ✓ · m106은 다음 세션 이월
커밋	966c93f "structure error responses as JSON with code and message (verified: application/json)" — 2 files, +11/-8 (ErrorResponse.java 신규)
★ 오늘의 깨달음	code가 왜 필요한가 — message 하나면 앱이 한글 문장을 contains로 검사해야 함(어제 Q2). code=불변 영문 상수(분기용), message=화면 문구. "새 문법이 아니라 설계 결정"이라 유추가 안 됐다
★ 눈으로 본 것	Content-Type: text/plain → application/json (세 에러 전부) = record→JSON 직렬화 = 7/7 마법 ④(@RequestBody)의 역방향 첫 가동
code 값 3개	ACCOUNT_NOT_FOUND · INVALID_AMOUNT · INSUFFICIENT_BALANCE (클래스명에서 따오되 IAE만 좁힘 — 아래 § 2)
태우 손코딩	ErrorResponse record 직접 작성, 교정 2곳(첫 필드 누락 / 대문자 Message)
별건	code-server : VPS에 브라우저 VS Code 띄워 폰 편집 시험 — 설치·8443 기동·폰 접속·폴더 열기 성공 (사지방 방화벽만 남은 변수)
정리	서버 돌(8080 ledger-api · 8443 code-server) 규율대로 내림, ss 빈 출력 ×2
다음	m106 CodeCrafters Redis 1단계 (다음 세션 프레시 스타트, 30~40분)

어제 표가 스스로 가리킨 다음 일을 오늘 했다 — 세 실패가 전부 400이라 클라이언트가 구분 못 하는 문제. 그걸 code(기계용)+message(사람용) 두 필드로 풀었다. 코드량은 record 한 줄 + 타입 교체 세 곳으로 적었지만, 오늘 시간을 쓴 곳은 코드가 아니라 **"code가 대체 왜 필요하냐"**였다 — message는 뜻에서 유추됐는데 code는 안 됐고, 그게 정확히 이게 문법이 아니라 설계라는 증거였다.

0. 오늘의 위치 — 예외 번역 장(7/12~15)의 완전 종결

날	한 일	응답의 모습
7/12	잔액부족 → 400 (핸들러 1개)	이제 실패: ... (text/plain)
7/13	심화 6문답 (지반)	—
7/14	핸들러 2개 추가 (3핸들러 체제)	이제 실패: ... ×3 (전부 400, text/plain)
7/15 (오늘)	에러 JSON 구조화	{"code": "...", "message": "..."} (application/json)

7/14에 "셋 다 400이라 구분 불가"를 눈으로 봤고(§ 6 관찰), 오늘 그걸 해소했다. 4일에 걸친 한 장이 닫혔다 — ledger-api가 이제 "정확한 상태코드 + 기계가 읽는 에러"를 갖췄다.

1. ErrorResponse record — 두 번째 record, 태우가 직접

작성한 풀코드 (**ErrorResponse.java** 전체)

```
package com.ledger.api;

public record ErrorResponse(String code, String message) {}
```

7/7의 TransferRequest 이후 두 번째 **record**다. 그때는 템플릿을 받았지만 오늘은 자리(빈칸)만 짚어주고 태우가 채웠다. 1일차 규칙(public record 이름 = 파일명) 그대로.

교정 2곳 (정직 기록)

#	첫 시도	문제	교정
①	<code>public record ErrorResponse(, String Message)</code>	첫 필드 누락 — 콤마 앞이 빈칸(code가 빠짐)	<code>String code</code> 채움
②	<code>String Message</code>	대문자로 시작 — 자바 관례는 필드명 소문자(대문자는 클래스명과 혼동, 컴파일은 되나 스타일 오류)	<code>String message</code>

②는 TransferRequest의 `fromId`, `toId` 가 전부 소문자로 시작했던 것과 같은 규칙 — 7/7에 이미 본 패턴이다. `cat` + `compileJava` 로 `public record ErrorResponse(String code, String message) {}` + BUILD SUCCESSFUL 확인.

2. ★ code가 왜 필요한가 — 오늘의 진짜 본체

태우의 반응이 정확했다: "**String Message**는 유추 가능했는데 **String Code**는 뜬금없다." 이 반응 자체가 중요한 신호다 — message는 "에러 메시지"라는 뜻에서 자연히 나오는데, code는 뜻이 아니라 **설계 결정**에서 나오기 때문이다.

message 하나로는 왜 안 되나

어제(7/14 §6) 관찰한 상태: 세 실패(없는 계좌·잘못된 금액·잔액부족)가 **전부 400**이고, 본문 메시지도 한글 문장이다. 이 API를 쓰는 앱이 "잔액 부족이면 충전 화면 띄우기"처럼 **경우별로 다르게 반응**하려면, 지금 유일한 방법은:

```
if (message.contains("잔액이 충분하지")) { 충전화면(); }
```

이게 왜 끔찍한가 (어제 Q2에서 이미 결론): - 오타 수정 한 번 → 앱이 조용히 깨진다 (에러도 안 남) - 번역 추가(영어 서비스) → 문장이 바뀌니 전부 깨진다 - 말투 순화("충분하지 않습니다" → "부족합니다") → 깨진다

문장은 **사람에게 보여주려고** 바뀌는 게 정상인데, 그 문장에 **기계 로직이 매달려** 있으면 사람 편의와 기계 안정성이 충돌한다.

code가 그 충돌을 없앤다

	code	message
대상	기계(앱 코드)	사람(화면)
예시	INSUFFICIENT_BALANCE	잔액이 충분하지 않습니다
변경	절대 안 바뀜(계약)	바뀌어도 됨(번역·순화 자유)
스타일	대문자 상수 (6/27 enum과 동일)	자연어 문장

앱은 `if (code.equals("INSUFFICIENT_BALANCE"))` 로 분기하고, `message` 는 화면에 그냥 띄운다. 이제 문장을 아무리 바꿔도 앱은 안 깨진다 — **분기의 기준(code)과 표시물(message)이 분리**됐기 때문이다.

→ **즉 이걸 새 자바 문법이 아니라 "왜 필드가 2개여야 하는가"의 답이었다.** code의 정체는 6/27에 enum 만들 때 이미 써본 "대문자, 불변, 기계용 이름" 스타일 그대로다. 낯선 건 문법이 아니라 그게 **설계에서 나온다는 것**이었다.

3. 핸들러 3개 수정 — <String> → <ErrorResponse>

3핸들러 동시 수정이라 EOF heredoc으로 한 번에 덮어썼다(7/7에 배운 이유: nano 저장 실수 위험 제거). 각 핸들러에서 두 곳씩, 총 6곳:

- 반환 타입: `ResponseEntity<String>` → `ResponseEntity<ErrorResponse>`
- body: `.body("이체 실패: " + e.getMessage())` → `.body(new ErrorResponse("코드값", e.getMessage()))`

완성된 TransferController 전문 (커밋 966c93f)

```

package com.ledger.api;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;

@RestController
public class TransferController {

    private final Bank bank = new Bank();

    public TransferController() {
        bank.createAccount("a", "이름", 2100);
        bank.createAccount("b", "이름", 3500);
        bank.createAccount("platform", "플랫폼", 0);
    }

    @PostMapping("/transfer")
    public String transfer(@RequestBody TransferRequest request) {
        bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
        return "a: " + bank.getAccount("a").getBalance()
            + ", b: " + bank.getAccount("b").getBalance()
            + ", platform: " + bank.getAccount("platform").getBalance();
    }

    @ExceptionHandler(InsufficientBalanceException.class)
    public ResponseEntity<ErrorResponse> handleInsufficientBalance(InsufficientBalanceException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse("INSUFFICIENT_BALANCE", e.getMessage()));
    }

    @ExceptionHandler(AccountNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleAccountNotFound(AccountNotFoundException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse("ACCOUNT_NOT_FOUND", e.getMessage()));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> handleIllegalArgument(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse("INVALID_AMOUNT", e.getMessage()));
    }
}

```

code 값 3개 — 왜 이렇게 지었나

예외	code 값	근거
InsufficientBalanceException	INSUFFICIENT_BALANCE	클래스명 그대로 (도메인 예외라 이름이 이미 정확)
AccountNotFoundException	ACCOUNT_NOT_FOUND	클래스명 그대로
IllegalArgumentException	INVALID_AMOUNT	★ 클래스명 안 씀 — JDK 공용이라 "IllegalArgumentException"는 너무 포괄적. "이 컨트롤러에서 실제로 뭐가 잘못됐나"(금액)를 code에 반영

세 번째가 7/14 § 1의 트레이드오프와 연결된다 — IllegalArgumentException은 JDK 공용이라 이 컨트롤러에서 amount 검증 말고 다른 데서도 터질 수 있다. 그래서 code 를 클래스명(ILLEGAL_ARGUMENT)이 아니라 **현재 이 자리의 실제 의미**(INVALID_AMOUNT)로 좁힌 것. 판정: @ExceptionHandler =3, BUILD SUCCESSFUL in 7s. cat으로 전문 재확인(폰 붙여넣기가 화면상 깨져 보였지만 실제 파일은 온전 — 눈으로 전체 확인 후 진행, "가정 말고 확인" 그대로).

4. ★ 검증 4연타 — Content-Type이 바뀌는 순간

부팅 고정 절차(grep Started 판정 → 그다음 curl) 지킴. Started in 4.767s 확인 후:

#	요청	상태	본문	Content-Type
①	없는 계좌(zzz)	400	<code>{"code": "ACCOUNT_NOT_FOUND", "message": "계좌를 찾을 수..."}</code>	application/json
②	amount -100	400	<code>{"code": "INVALID_AMOUNT", "message": "출금액은 0보다 커..."}</code>	application/json
③	amount 99999	400	<code>{"code": "INSUFFICIENT_BALANCE", "message": "잔액이 충분..."}</code>	application/json
④	정상 이체(회귀)	200	<code>a: 1080, b: 4500, platform: 20</code>	text/plain

★ 오늘의 관련 포인트가 정확히 찍혔다: 어제까지 `text/plain` 이던 Content-Type이 세 에러 전부 `application/json` 으로 바뀌었다.

이게 왜 중요한가 — 마법 ④의 역방향

7/7에 배운 마법 ④(@RequestBody)는 **JSON → 객체**(역직렬화)였다:

```
curl이 보낸 JSON 텍스트 → 객체 → TransferRequest record
```

오늘은 정확히 반대 방향이 처음 가동됐다:

```
new ErrorResponse(...) 객체 → 객체 → JSON 텍스트 + Content-Type: application/json
```

`body()` 에 String을 넣으면 객체가 할 일이 없어 그대로 text/plain으로 나갔는데(어제까지), **record(객체)를 넣으니 객체가 "이건 JSON으로 바꿔야겠다"** 판단해서 직렬화하고 헤더까지 application/json으로 바꾼 것. **같은 마법(객체)의 왕복 두 방향을 이제 다 봤다** — 들어올 때(@RequestBody)와 나갈 때(return 객체).

(터미널에 본문이 `계좌를 찾을 수` 처럼 끊겨 보인 건 다음 명령이 같은 줄에 이어져 화면상 잘린 것 — 실제 응답이 잘린 게 아니다. code 매칭 정확, message도 도메인 문장 시작 일치.)

부수 관찰 — 정상 이체는 아직 text/plain

④(정상, 200)는 여전히 `text/plain` 이다. 오늘은 **에러 경로만** 손냈으니 당연한 결과지만, 성공 응답도 구조화하면 API 전체가 일관된 JSON이 된다. **다음 조각 후보로만** 기록 (지금 할 필요 없음).

5. 별건 — code-server 편집환경 시험 (성공)

왜 이걸 했나

폰으로만 타이핑하니 VPS 화면(nano)이 좁아 불편하다는 문제. 태우가 "PC에서 타이핑 → 네이버 내게쓰기 → 폰에 복붙" 안을 냈는데, 여기엔 제약이 걸린다 — **사지방(부대 공용 PC)은 GitHub 로그인인 안 된다**(github.dev·웹에디터 안 죽음). 그래서 대안: **VPS 자체에 브라우저용 VS Code(code-server)를 띄우면** 사지방 브라우저로 내 서버에 접속해 편집 — GitHub 로그인도, 메일 다리도 필요 없다.

시험 절차와 판정

```
curl -fsSL https://code-server.dev/install.sh | sh # 4.128.0 설치
code-server --version && ufw status # → 4.128.0 / Status: inactive (방화벽 안 막음)
```

설정 수정(`~/.config/code-server/config.yaml`): `bind-addr` 을 기본 `127.0.0.1:8080` (로컬 전용) → `0.0.0.0:8443` (외부 접속 + 8080은 Tomcat과 충돌 회피). 비번은 자동생성값 유지(grep -c로 줄 존재만 확인, 값은 화면에 안 띄움).

```
code-server > code-server.log 2>&1 &
grep "HTTP server listening" code-server.log # → HTTP server listening on http://0.0.0.0:8443/
```

폰 사파리 → `http://104.236.232.210:8443` : - 로그인 화면 → 비번 입력 → 통과 - code-server 시작 화면(New File / Open Folder / Clone Git Repo) 도달 - File → Open Folder → `/root/ledger-api` → **폴더 열림, 파일 목록 보임**

→ **시험 성공**. 설치·기동·폰 접속·인증·폴더 열기까지 전 구간 검증. 다음 사지방 때 이 주소로 바로 편집 가능하다는 뜻 — **남은 변수는 사지방 방화벽이 8443을 막느냐 하나뿐** 이다(부대망은 밖으로 나가는 포트를 제한할 수 있음).

보안 규율

code-server는 **공개 IP에 떠 있는 에디터**라 ledger-api보다 오히려 더 민감하다(누구나 접속 시도 가능). 그래서 강한 비번(자동생성) + **"쓸 때만 켜고 끝나면 내린다"** 규율을 그대로 적용 — 오늘 시험 끝나고 즉시 내렸다.

6. 정리 — 서버 둘 규율대로 내림

```
kill %1 # ledger-api(8080)
sleep 3 && ss -tlnp | grep 8080 # → 빈 출력 (Terminated)
pkill -f code-server # code-server(8443)
sleep 2 && ss -tlnp | grep 8443 # → 빈 출력
```

공개 IP에 뜬 서버는 **ss 빈 출력**으로 내림을 판정(7/7 이후 규율). 오늘은 8080·8443 둘 다 확인 — 특히 code-server(에디터)를 켜둔 채 끝내지 않은 게 중요.

커밋:

```
[main 966c93f] structure error responses as JSON with code and message (verified: application/json)
2 files changed, 11 insertions(+), 8 deletions(-)
create mode 100644 src/main/java/com/ledger/api/ErrorResponse.java
e567011..966c93f main -> main
```

7. 로드맵 위치

- 예외 번역 장(7/12~15) 완전 종결 — 3핸들러가 구조화 JSON({code, message}) 응답.
- ledger-api 현재: GET /health · POST /transfer — 멍등성 + 복식부기 + 세 실패 정확한 4xx + **기계가 읽는 에러 JSON**. 도메인 수정 여전히 0줄.
- 편집환경: code-server VPS에 상주 준비 완료(8443), 사지방 방화벽만 다음 확인.
- 7월 잔여(16일): m106 CodeCrafters Redis(다음 세션 프레시 스타트, 30~40분) → m107 월말 점검. 관문1은 사실상 조기 충족, 2주 남기고 완주 해도.
- 8월 WAL 이유 3개 유지: 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

8. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 에러 응답에 code와 message 두 필드가 필요한 이유는? message 하나면 앱이 뭘 해야 하고, 왜 그게 끔찍한가?
2. code와 message의 차이를 "변경"의 관점에서 대비하라 — 뭐가 바뀌어도 되고 뭐가 안 바뀌어야 하나?
3. IllegalArgumentException의 code를 **ILLEGAL_ARGUMENT** 가 아니라 **INVALID_AMOUNT** 로 지은 이유는? (7/14 트레이드오프와 연결)
4. Content-Type이 text/plain → application/json으로 바뀐 원인은? body()에 String과 record(객체)를 넣을 때 잭슨의 행동이 어떻게 다른가?
5. 이것이 7/7 @RequestBody(마법 ④)와 어떤 관계인가? "왕복 두 방향"이 각각 뭔가?
6. 정상 이체(200)는 왜 아직 text/plain인가? 이걸 바꾸려면 뭘 해야 하나?
7. ErrorResponse record에서 필드명을 소문자로 시작해야 하는 이유는?
8. code-server를 8080이 아니라 8443에 띄운 이유는? bind-addr을 127.0.0.1이 아니라 0.0.0.0으로 바꾼 이유는?
9. code-server에 "쓸 때만 켜고 끝나면 내린다" 규율이 ledger-api보다 더 중요한 이유는?
0. 사지방에서 이 편집환경을 쓸 때 남은 단 하나의 변수는 무엇인가?

9. 회고

코드는 적었지만 이해가 컸던 날이다. record 한 줄, 타입 교체 세 곳 — 손이 움직인 양은 30분도 안 걸릴 분량이었다. 그런데 시간을 쓴 곳은 "code가 대체 왜 필요하냐"였고, 그 질문이 나온 것 자체가 오늘의 핵심이었다. message는 "에러 메시지"라는 뜻에서 자연히 유추됐는데 code는 안 됐다 — "뜨금없다"는 반응이 정확했던 건, 이게 문법이 아니라 설계라서다. 어제 Q2에서 "문자열 분기는 끔찍하다"를 논리로 알았고, 오늘 그 해법(code)을 직접 만들면서 왜 두 필드여야 하는지를 몸으로 이었다. code=기계용 불변, message=사람용 가변. 이 분리가 오늘 남는 한 줄이다.

눈으로 본 것도 값졌다. Content-Type이 text/plain에서 application/json으로 바뀌는 순간 — body에 String을 넣던 어제와 record를 넣는 오늘의 유일한 차이가, 잭슨이 직렬화를 하느냐 마느냐를 갈랐다. 7/7에 JSON이 record가 되는 걸(마법 ④) 봤는데, 오늘 record가 JSON이 되는 역방향을 봤다. 같은 마법의 왕복을 다 본 셈이다. 이걸 설명으로 들은 게 아니라 헤더 한 줄이 바뀐 걸로 확인했다 — 가정 말고 확인, 오늘도 유지됐다.

정직하게 남길 것 둘. 하나 — 부팅 절차를 지켰다. 7/14에 "성급한 curl 재발"을 고정 절차로 못박았는데, 오늘은 grep Started 판정을 먼저 하고 curl로 갔다. 실수를 규칙으로 바꾼 게 하루 만에 지켜졌다. 둘 — record 교정 2곳(첫 필드 누락, 대문자 시작)은 사소하지만, cat으로 전체를 다시 확인하고 넘어간 습관이 폰 붙여넣기 깨짐(화면상)에도 안 흔들리게 해줬다. 숫자(grep=3)만 믿지 않고 눈으로 전문을 본 것.

그리고 오늘은 코드 밖에서도 하나 풀었다. 폰 편집의 불편을 code-server로 우회하는 다리를 봤다 — 사지방 GitHub 로그인 불가라는 제약을 정면으로 받고, VPS에 내 에디터를 띄우는 방식으로 돌아갔다. 방화벽 하나만 남았으니 다음 사지방 때 확인하면 된다. 편집은 브라우저로, 실행·검증은 여전히 폰 Termius로 — 역할이 갈리지만 그게 지금 환경의 최선이고, 근본 해결(블루투스 키보드)은 건의서에 있다.

예외 번역 장이 이걸로 닫혔다. 다음은 완전히 다른 걸 — m106, 내 손으로 6379 포트를 여는 것. 7/6에 "마법 목록 ①"로 올렸던 그것(Tomcat이 8080을 연다)의 아래층을, 이번엔 직접. 8월 저수준의 워밍업이고, 이 포폴의 정체성 그 자체다.

한 줄: 에러 응답을 {code, message} JSON으로 구조화(966c93f) — code=기계용 불변, message=사람용 가변의 분리를 직접 만들며 왜 두 필드인지를 이었고, Content-Type이 application/json으로 바뀌는 걸로 record→JSON 직렬화(마법 ④ 역방향)를 눈으로 확인. 예외 번역 장 종결. 덤: code-server로 폰 편집 다리 놓기 성공.